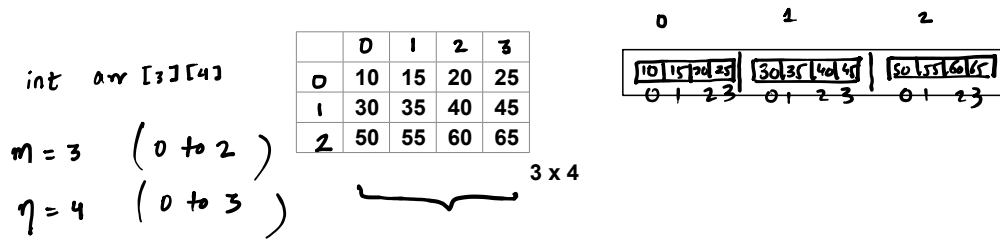


Introduction to 2D Arrays

A 2D-array is an **array of 1D arrays**, referred by a single name, which is used to store a **sequence** of values of the **same type** and can be visualized as a **matrix**.



How to **declare** a 2D-array in C++ ?

$$12 \cdot 4B = 48B \text{ (contiguous)}$$

int arr[3][4];

```
// syntax for a 2D-array declaration
type name[rows][cols];
```

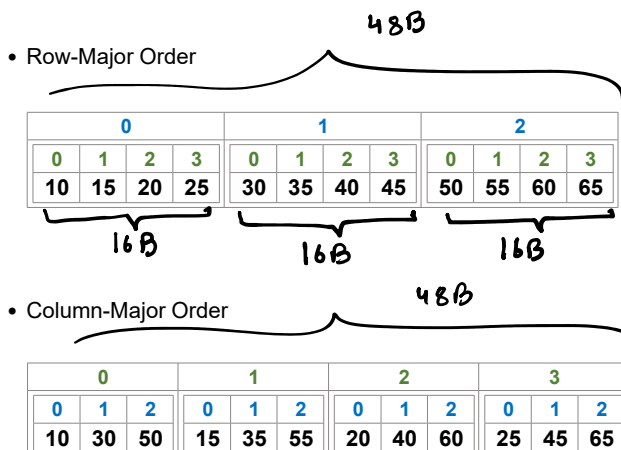
A 2D-array is allocated a **contiguous** block of memory to store its elements.

	0	1	2	3
0	10	15	20	25
1	30	35	40	45
2	50	55	60	65

3 x 4

$$12 \text{ int} \times 4B = 48B$$

This allocation of contiguous memory is done either in the **row-wise** manner a.k.a row-major order or in a **column-wise** manner a.k.a column-major order.

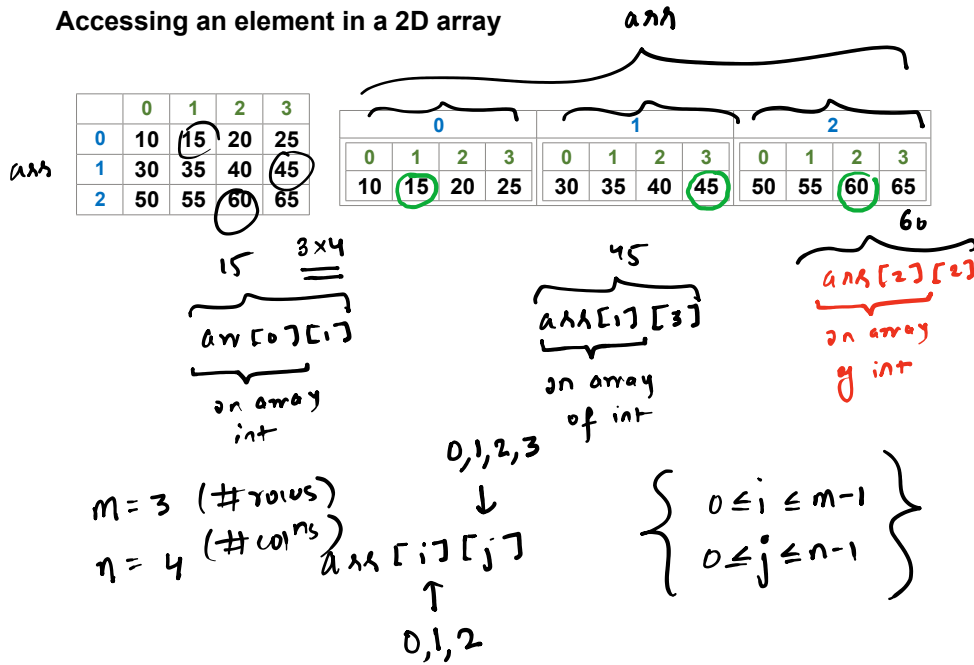


	0	1	2	3
0	10	15	20	25
1	30	35	40	45
2	50	55	60	65

3 x 4

12B 12B 12B 12B

Accessing an element in a 2D array



⇒ Since 2D-arrays are allocated memory in a **contiguous** manner, accessing an element in a 2D-array is a **constant time** i.e. **O(1)** operation.

2D-Array Initialization

By default, when we declare a 2D-array in the local-scope, it contains **garbage** value.

⇒

```
int arr[3][4] = {{10, 15, 20, 25},
                 {30, 35, 40, 45},
                 {50, 55, 60, 65}};
```

	0	1	2	3
0	10	15	20	25
1	30	35	40	45
2	50	55	60	65

arr

3x4

During the initialization of a 2D-array, specifying the no. of rows is **optional**.

While initializing a row in a 2D-array, the size of the **initializer list** must not exceed the no. of columns in the 2D-array.

```
int arr[][2] = {{10, 20, 30},
                {40, 50}};
```

error

What if the size of the row initializer list is less than the no. of columns ?

```
int arr[][3] = {{10, 20, 30},
                {40, 50}};
```

uninitialized colⁿ will be filled with zeros.

`int arr[3][4] = {};`
 or
`{0};`
 or
`memset(arr, 0, sizeof(arr));`

$i = 0, 1, 2$ ↓

		0	1	2	3
0	10	15	20	25	
1	30	35	40	45	
2	50	55	60	65	

$m = 3$

for ($i = 0; i < m; i++$) {

`arr[i][0]`
`arr[i][1]`
`arr[i][2]`
`arr[i][3]`

}

`arr[0][0]`
`arr[0][1]`
`arr[0][2]`
`arr[0][3]`

0th row ←

`arr[1][0]`
`arr[1][1]`
`arr[1][2]`
`arr[1][3]`

1st row ←

`arr[2][0]`
`arr[2][1]`
`arr[2][2]`
`arr[2][3]`

2nd row ←

⇒ Reading Values in 2D-array

$m \leq 10$
 $n \leq 10$

constraints

`int arr[10][10];` // based on constraints

`int m, n;`

`(cin >> m >> n);`

⇒

0	.	.	.	.
1	.	.	.	.
2	.	.	.	.
3				
4				
5				
6				
7				
8				
9				

10 x 10

$m = 3$

$n = 4$

for ($i = 0; i < m; i++$) {
 for ($j = 0; j < n; j++$) {
 `cin >> arr[i][j];`
 }
 }

Wave Print

Wave Print

Given an integer matrix of dimensions $m \times n$, write a program that prints the matrix in the **wave** form.

Example

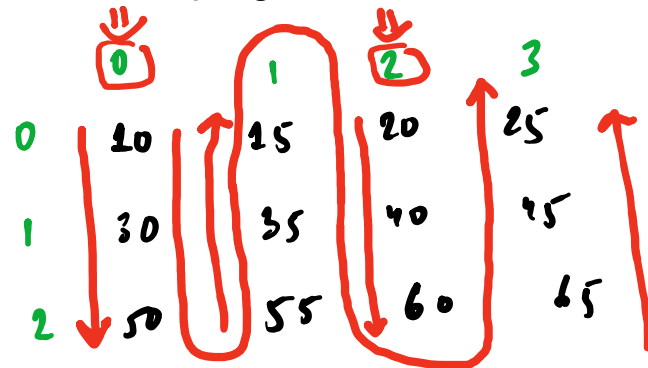
Input

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

Output

10	40	70	80	50	20	30	60	90
----	----	----	----	----	----	----	----	----

3x3



Transpose a Matrix

Transpose a Square Matrix

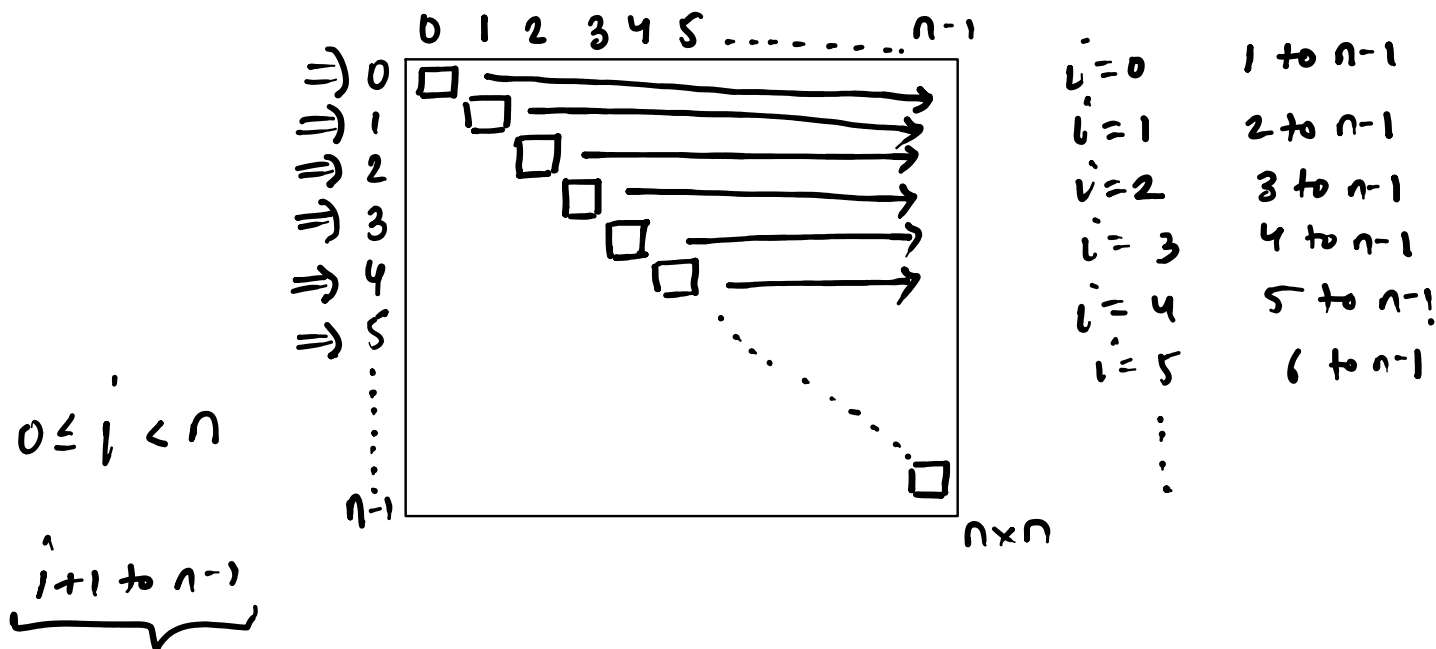
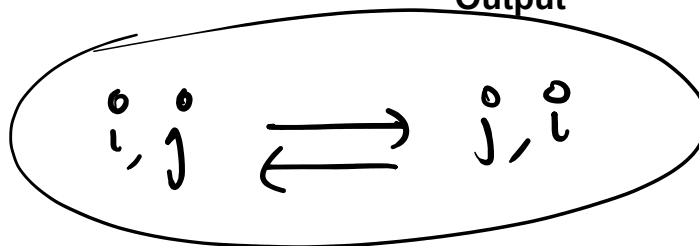
Given an square matrix of integers of dimensions $n \times n$, write a program to **transpose** it.

	0	1	2	3
0	11	12	13	14
1	15	16	17	18
2	19	20	21	22
3	23	24	25	26

Input

	0	1	2	3
0	11	15	19	23
1	12	16	20	24
2	13	17	21	25
3	14	18	22	26

Output



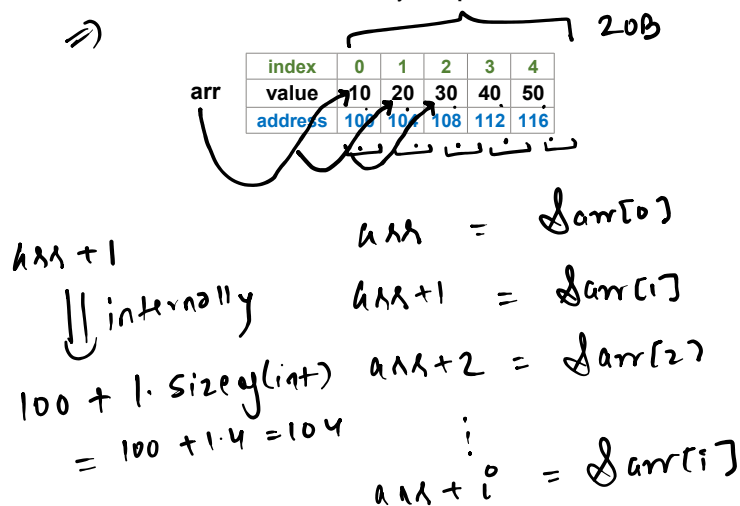
[HW] Rotate Image

<https://leetcode.com/problems/rotate-image/>

`int arr[] = {10, 20, 30, 40, 50}`

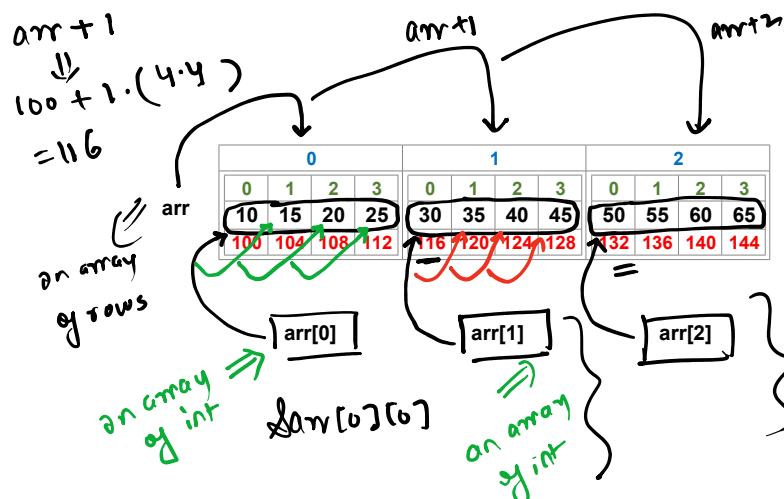
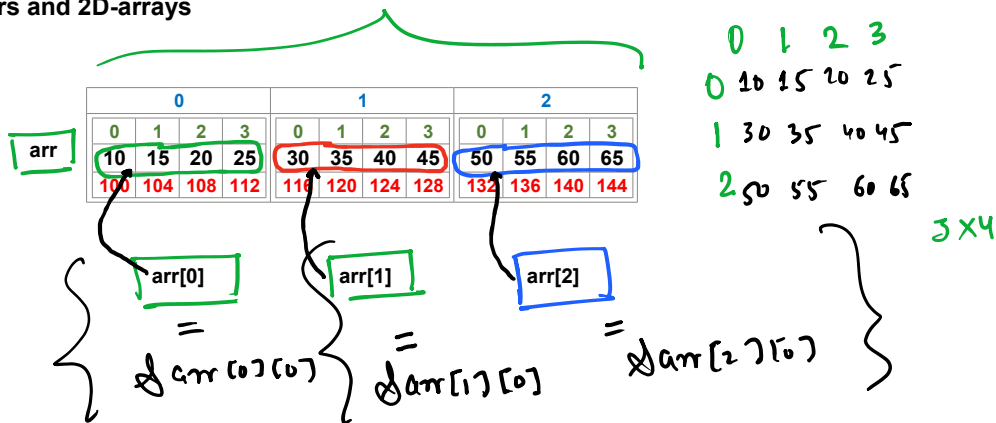
Pointers and 1D-arrays

In C++, we can think of **name** of an array as a **pointer** to the element at the **0th** index.



$*arr = arr[0]$
 $*(arr+1) = arr[1]$
 $*(arr+2) = arr[2]$
 $\vdots$
 $*(arr+i) = arr[i]$
↓
 $arr + i * \text{sizeof(int)}$

Pointers and 2D-arrays



$$arr[2] + 3$$

$$\downarrow$$

$$132 + 3 \cdot 4 = 144$$

$$arr[0] + 2$$

$$\downarrow$$

$$100 + 2 \cdot 4 = 108$$

$$arr + 2$$

$$\downarrow$$

$$100 + 2 \cdot (4 \cdot 4) = 132$$

$$arr[i] + j$$

internally

an array of int

$$arr[i] + j * \text{sizeof(int)}$$

internally

$$arr + i \cdot \text{sizeof(row)}$$

of int
 $\underline{arr + i}$
 $\Downarrow$
 an array
 of rows

internally $\Rightarrow$ $\overbrace{arr + i \cdot \text{size}(\text{row})}^{\text{# rows} * \text{size}(\text{row})}$
 $\underline{\underline{\text{size}(\text{int})}}$

$arr[i][j]$

1. go to the i^{th} row

$$\Rightarrow \boxed{arr + i}$$

2. go inside the i^{th} row

$$* (arr + i)$$

$=$

$$arr[i]$$

3. go to the j^{th} col

$$\Rightarrow \boxed{arr[i] + j}$$

4. go inside the j^{th} col

$$* (arr[i] + j)$$

$=$

$$arr[i][j]$$

Matrix Search

Matrix Search

Given an integer matrix of dimensions $m \times n$, and a target integer T , write a program to search for the target in the given matrix.

Example

Input : $T = 10$

	0	1	2
0	50	80	20
1	90	10	70
2	60	30	40

$t = 100$
o/p: false

Output : True

Sorted Matrix Search I

Given an integer matrix of dimensions $m \times n$, which is **sorted row-wise** and a **target** integer T , write a program to **search** for the target in the given matrix.

Example

Input : $T = 50$

	0	1	2
0	40	50	60
1	10	20	30
2	70	80	90

Output : True

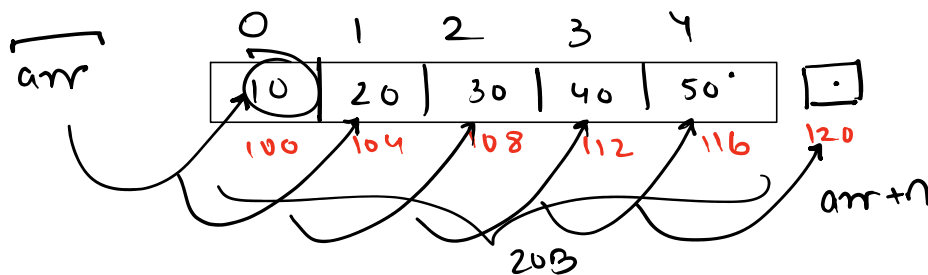
iterate over rows,

apply binary search
on each row to search
for t

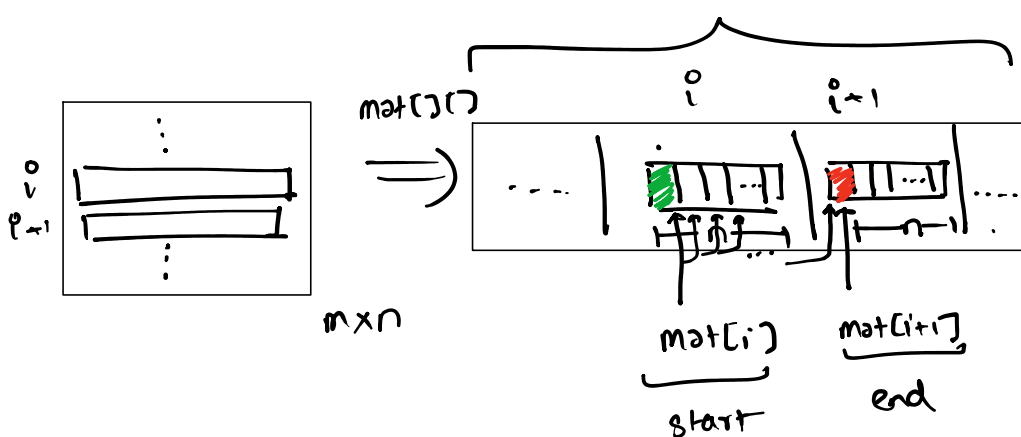
$m \cdot \log n$

$\text{int arr}[] = \{ 10, 20, 30, 40, 50 \}$

$\text{int } n = 5$



binary-search(arr, arr+n, t) $[0, n)$

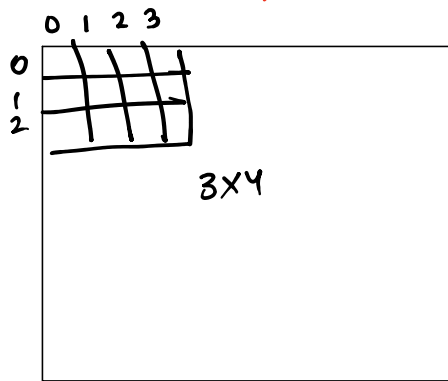


~~start = mat[i]~~

✓
 $\text{mat}[i+n]$

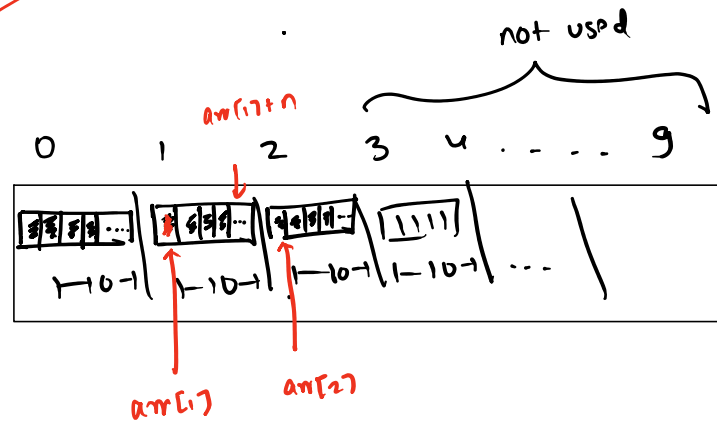
start = mat[i]

end = ~~mat[i+1]~~ or $\boxed{\text{mat}[i] + n}$



10x10

$\Rightarrow$



Sorted Matrix Search II

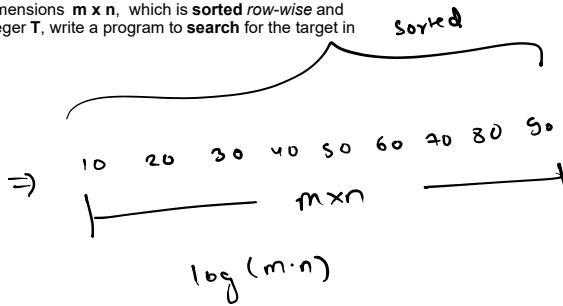
Given an integer matrix of dimensions $m \times n$, which is **sorted row-wise** and **column-wise** and a **target** integer T , write a program to **search** for the target in the given matrix.

Example

Input : $T = 50$

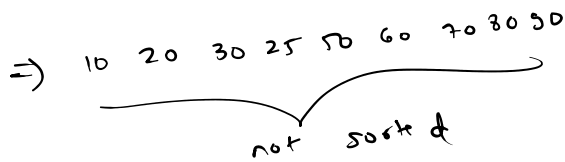
	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

Output : True

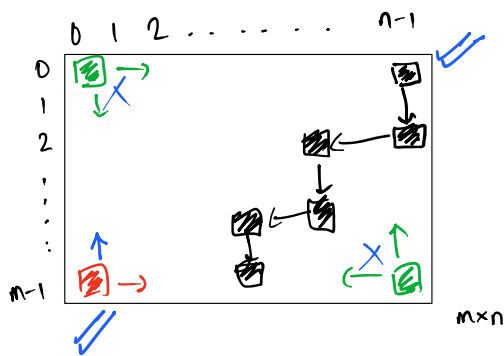


$t = 55$

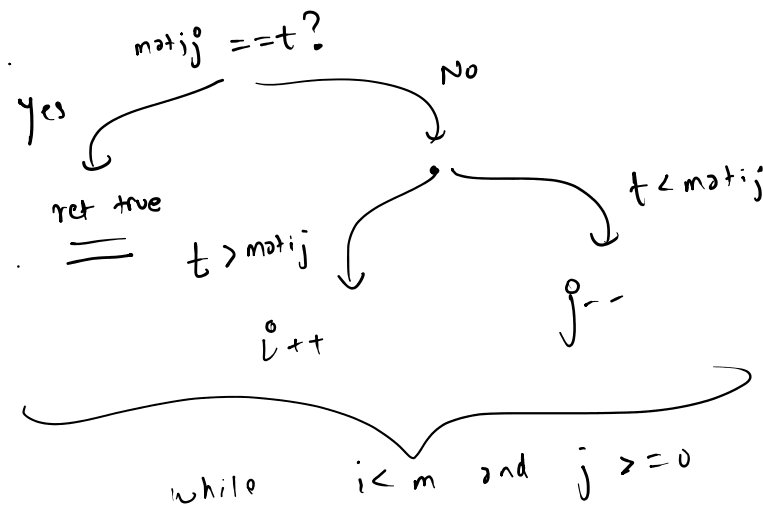
	0	1	2
0	10	20	30
1	25	50	60
2	70	80	90



Staircase Search



$i = 0$ $j = n-1$



[HW] Count Negative Numbers in Sorted Matrix

<https://leetcode.com/problems/count-negative-numbers-in-a-sorted-matrix/>

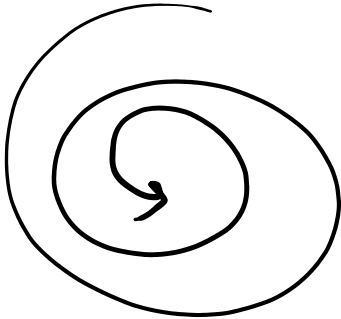
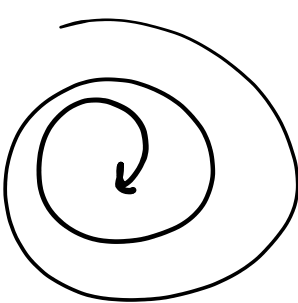
Spiral Print a Matrix

Given an integer matrix of dimensions **m x n**, design an algorithm to print the matrix spirally in a **clock-wise** direction.

Example

Input

	0	1	2	3
0	11	12	13	14
1	22	23	24	15
2	21	26	25	16
3	20	19	18	17



Output

11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

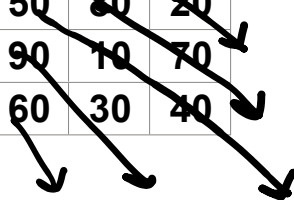
	0	1	2	3
0	11	12	13	14
1	22	23	24	15
2	21	26	25	16
3	20	19	18	17

Diagonal Sort a Matrix

Given an integer matrix of dimensions **m x n**, write a program to **sort** it **diagonally**.

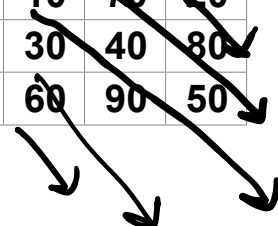
Input

	0	1	2
0	50	80	20
1	90	10	70
2	60	30	40



Output

	0	1	2
0	10	70	20
1	30	40	80
2	60	90	50



[HW] <https://cses.fi/problemset/task/1652>

2D Prefix Sum

You are given a 2d-array of $M \times N$ integers, and Q queries, each of the form (l_i, r_i) and (l_j, r_j) where $0 \leq l_i, r_i \leq l_j, r_j \leq N-1$. Your task is to find for each of the Q queries sum of all elements within the **submatrix** defined by the top-left corner (l_i, r_i) and the bottom-right corner (l_j, r_j) .

Example

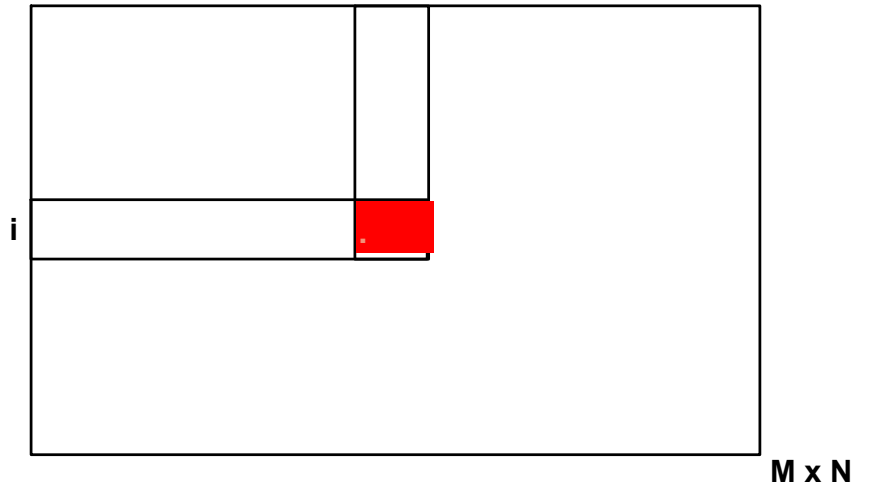
N = 5

	0	1	2	3	4
0	2	3	0	1	5
1	4	7	9	3	2
2	6	8	1	0	4
3	9	5	4	2	3
4	0	6	8	6	1

Q = 3

Q_0	0	0	1	1
Q_1	1	2	3	4
Q_2	2	0	4	1

j



	0	1	2	3	4
0	2	3	0	1	5
1	4	7	9	3	2
2	6	8	1	0	4
3	9	5	4	2	3
4	0	6	8	6	1

	0	1	2	3	4
0					
1					
2					
3					
4					

j

